

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа 3

Выполнил:

Баранов Владимир

Группа

J3213

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

В рамках данной лабораторной работы было предложено мигрировать ранее разработанное приложение (в рамках ЛР1 и ЛР2) на фреймворк Vue.js. Тема проекта остаётся прежней – PipelineLab, платформа для управления ML-пайплайнами (аналог MLflow): вход, регистрация, личный кабинет, поиск экспериментов с фильтрацией, страница эксперимента (логи, метрики, артефакты), управление моделями.

Требования к Vue-приложению:

- Подключённый роутер (Vue Router).
- Работа с внешним API (через axios).
- Разумное деление на компоненты – демонстрация компонентного подхода.
- Использование composables для выделения повторяющегося функционала в отдельные файлы.

Ход работы

Существующий многостраничный сайт ЛР1 (HTML/CSS/Bootstrap + ванильный JS, json-server в качестве бэкенда) был переписан в виде одностраничного приложения на Vue 3. Бэкенд (json-server + json-server-auth) и схема данных (users, experiments, metrics, artifacts, models) сохранены, чтобы изменения касались только фронтенда.

Стек и инструментарий

Использованный стек:

- Vue 3 – Composition API, синтаксис <script setup>.
- Vite – dev-сервер с hot reload и сборка production-бандла.
- Vue Router 4 – клиентский роутинг в режиме history.
- axios – HTTP-клиент с настроенным interceptor для авторизации.

- json-server + json-server-auth – REST-бэкенд.
- Bootstrap 5.3.2 -- подключён через CDN

Структура проекта

Маршрутизация (Vue Router)

Подключён Vue Router 4 в режиме history (createWebHistory). Все ранее отдельные html-страницы стали маршрутами одного SPA:

- Главная (HomeView).
- /login, /register – Вход и Регистрация (только для неавторизованных).
- /dashboard – Личный кабинет.
- /experiments – Список экспериментов с фильтрами.
- /experiments/:id – Страница конкретного эксперимента.
- /models – Реестр моделей.

Для маршрутов, требующих авторизации, в meta указан флаг requireAuth, для страниц входа/регистрации – guestOnly. Глобальный navigation guard router.beforeEach проверяет наличие токена через composable useAuth и при отсутствии редиректит на /login (с сохранением целевого пути в query.redirect).

Работа с внешним API через axios

В файле src/api/client.js создаётся инстанс axios.create({ baseURL: 'http://localhost:3000' }). К нему добавлен request-interceptor, который читает токен из localStorage и подставляет его в заголовок Authorization: Bearer <token>. Все запросы к защищённым ресурсам (/600/experiments, /600/metrics, /600/artifacts, /600/models) проходят через этот клиент. Регистрация и вход выполняются POST-запросами на /register и /login (json-server-auth).

Composables

Повторяющаяся логика вынесена в отдельные файлы из папки `src/composables/`:

- `useAuth` – singleton-стейт авторизации (`token`, `user`) на `reactive()`, методы `login/register/logout`, `computed`-свойства `isAuthenticated`, `currentUserId`, `currentUsername`. Состояние сохраняется в `localStorage`.
- `useTheme` – переключение между темами `lab` (светлая) и `pipeline` (тёмная). Сохранение выбора в `localStorage`, синхронизация атрибутов `data-theme` и `data-bs-theme` на `html` через `watchEffect`.
- `useToast` – стек неблокирующих уведомлений вместо `alert()`. Методы `success/error/info`, автоматическое скрывание через таймер. Используется во всех модулях, где нужна обратная связь.
- `useExperiments` – CRUD-операции для экспериментов (`fetchAll`, `fetchById`, `create`, `update`, `remove`), функция `applyFilters` для клиентской фильтрации по дате/метрике/статусу/тегу, вспомогательные функции `normalizeStatus` и `normalizeLog` (для совместимости со старыми данными ЛР1, где логи хранятся как строки), фабрика `makeLog` для структурированных логов.
- `useModels` – CRUD-операции моделей, выборка активных моделей для `select`-поля при создании эксперимента.
- `useTableSort` – универсальный `composable` сортировки таблиц по клику на заголовок колонки. Возвращает `sorted (computed)`, функцию `toggle` и `helper headerClass` для подсветки направления.

Компонентный подход

Интерфейс декомпозирован на самостоятельные `.vue`-компоненты с четкими границами ответственности:

- Layout-компоненты (AppNavbar, ThemeToggle) – переиспользуются на всех страницах через App.vue.
- UI-примитивы (BaseModal, StatusBadge, EmptyState, LoadingSpinner, ToastContainer, Icon) — не привязаны к доменной модели и используются в нескольких местах.
- Доменные компоненты экспериментов: ExperimentFilters (фильтры с v-model), ExperimentTable(таблица с поиском и сортировкой), ExperimentForm (форма создания), MetricsPanel, ArtifactsPanel, LogsPanel.
- Доменные компоненты моделей: ModelTable, ModelForm.
- Связь между компонентами – через props (вниз) и emit-события (вверх). Глобальное состояние (авторизация, тема, тосты) – через composables.

Вывод

В ходе лабораторной работы существующий проект PipelineLab был мигрирован с многостраничного ванильного HTML/JS на Vue 3 приложение. Все требования задания выполнены: подключён Vue Router (history-режим, navigation guard на авторизацию), работа с API реализована через axios с interceptor для Bearer-токена, интерфейс декомпозирован на самостоятельные компоненты (layout, UI, доменные), повторяющаяся логика вынесена в composables (useAuth, useTheme, useToast, useExperiments, useModels, useTableSort).